

Reviewer Pack — Magnus Level-2 Defect Bounds DNF_min via Truth-Table Inversi...

Entry 8bf8c5c64dba · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-28 13:27:06 UTC

Magnus Level-2 Defect Bounds DNF_min via Truth-Table Inversions

Entry ID: 8bf8c5c64dba **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-28 13:27:06 UTC

1. Conjecture

Field A (mathematical branch): Magnus expansion / Chen’s iterated-integral signature in non-commutative formal power series of free groups (Magnus 1935; Chen 1957; Reutenauer, Free Lie Algebras 1993; Lyons rough-path tradition). The level-2 ‘inversion defect’ $\lambda_2(w) = |\#\{i < j : w_i=0, w_j=1\} - \#\{i < j : w_i=1, w_j=0\}|$ is exactly the pairing of the truncated Magnus signature $S^{\leq 2}(w) \in Q\langle\langle A, B \rangle\rangle$ with the commutator $[A, B]$, a free-Lie-algebra invariant computable in one $O(|w|)$ pass and rarely deployed in meta-complexity. **Field B** (complexity object): $\text{DNF_min}(f)$, the minimum number of product terms in a DNF representation of a Boolean function $f: \{0,1\}^n \rightarrow \{0,1\}$ presented by its 2^n -bit truth table — the Hirahara-Oliveira-Santhanam restricted-MCSP proxy used in meta-complexity worst-case-to-average-case reductions and the canonical truth-table-input meta-complexity object.

Statement:

Let $T = \text{TT}(f) \in \{0,1\}^{2^n}$ be the lex-ordered truth table of f and define $\lambda_2(f) = |\text{asc}(T) - \text{desc}(T)|$ where $\text{asc}(T) = \#\{i < j : T_i=0, T_j=1\}$ and $\text{desc}(T) = \#\{i < j : T_i=1, T_j=0\}$. Then for every Boolean f on n variables, $\log_2(1 + \lambda_2(f)) \leq n + \log_2(1 + \text{DNF_min}(f)) + 5 \cdot \log_2(n+1)$. A single f violating this inequality refutes the conjecture; for uniformly random f one has $\log_2 \lambda_2(f) \approx 1.5 n$ by anti-concentration, so the bound is exponentially below random and forces $\text{DNF_min}(f) = 2^{\Omega(n)}$ only on the trivial counting fraction.

Rationale (proposer’s reasoning):

The Magnus expansion linearizes a binary word into a free Lie algebra; its level-2 $[A, B]$ -coefficient is the inversion-pair imbalance of the truth-table string, computable in a single $O(2^n)$ sweep. Low-DNF functions have truth tables decomposing as the union of M structured sub-rectangles whose individual inversion imbalances align in a near-cancelling way, so λ_2 should grow like $M \cdot 2^n$ rather than the random $2^{1.5n}$. The hardness predicate ‘ $\lambda_2(f) \geq 2^{n+\log M+O(\log n)}$ ’ is satisfied by a $1 - \exp(-\Omega(n))$ fraction of random f only as a trivial counting bound, and the small- λ_2 side (which is the structural side actually used to upper-bound DNF_min) holds for an exponentially tiny fraction of random f , dodging the Razborov-Rudich largeness condition and hence the natural-proofs barrier.

Taxonomy category: META_COMPLEXITY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 68c92cf285d80822

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across all $n \in \{8, 10, 12, 14, 16, 18, 20\}$ and all four ensembles (uniform random, planted k-DNF, symmetric/threshold, random LTF) with ≥ 30 seeds per $(n, \text{ensemble}, M)$ cell (≥ 840 instances total), every instance must satisfy $\log_2(1 + \lambda_2(f)) \leq n + \log_2(1 + \text{DNF_min_or_upper}(f)) + 5 \cdot \log_2(n+1)$; a single violation falsifies.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL PROOFS	SAFE	0.78	SAFE	SAFE
ALGEBRIZATION	SAFE	0.85	SAFE	SAFE
KARP_LIPTON	SAFE	0.90	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Magnus expansion signature Boolean function DNF complexity - iterated integral signature truth table meta-complexity MCSP-free Lie algebra commutator inversions DNF minimization lower bound

Top relevant hits considered: - [<http://arxiv.org/abs/0810.5488v1>] The Magnus expansion and some of its applications - [<http://arxiv.org/abs/1501.01331v1>] DNF complexity of complete boolean functions - [<http://arxiv.org/abs/1407.6169v3>] Multiplicative Complexity of Vector Valued Boolean Functions - [<http://arxiv.org/abs/1509.08963v3>] Integrals of Lipschitz-Hankel Type, Legendre Functions, and Table Errata - [<http://arxiv.org/abs/2402.02935v4>] Nuclear mass table in deformed relativistic Hartree-Bogoliubov theory in continuum, II: Even- Z nuclei - [<http://arxiv.org/abs/1302.0560v1>] X-ray behaviour of GRBs detected by INTEGRAL/JEM-X - [<http://arxiv.org/abs/1212.5380v2>] On properties of principal elements of Frobenius Lie algebras - [<http://arxiv.org/abs/2411.11095v3>] Invariant theory and coefficient algebras of Lie algebras

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=146.8s

5.1 Generated Python source

```
import random
import math
import sys
import json
from itertools import product

def truth_table(f, n):
    return [f(i) for i in range(2**n)]
```

```

def lambda_2(T):
    asc = desc = 0
    for i in range(len(T)):
        if T[i] == 0:
            asc += sum(T[j] for j in range(i+1, len(T)) if T[j] == 1)
        else:
            desc += sum(T[j] for j in range(i+1, len(T)) if T[j] == 0)
    return abs(asc - desc)

def quine_mccluskey(f, n):
    # Simplified version of Quine-McCluskey algorithm
    minterms = [i for i in range(2**n) if f(i)]
    active = minterms.copy()
    prime_implicants = []
    while len(active) > 1:
        pairs = [(active[i], active[j]) for i in range(len(active)) for j in range(i+1, len(active))]
        new_active = []
        for pair in pairs:
            diff_count = sum(1 for a, b in zip(pair[0].to_bin(), pair[1].to_bin()) if a != b)
            if diff_count == 1:
                new_term = (pair[0] & pair[1]) << 1
                if new_term not in new_active:
                    new_active.append(new_term)
        active = new_active
    prime_implicants.extend(active)
    return len(prime_implicants)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [8, 10, 12, 14]
    results = []

    for n in n_values:
        for ensemble_type in ["random", "planted", "symmetric", "LTF"]:
            if ensemble_type == "random":
                f = lambda x: random.choice([0, 1])
                DNF_min_or_upper = 2**n
            elif ensemble_type == "planted":
                M = 2**(n-2)
                f = lambda x: sum((x >> i) & 1 for i in range(n)) >= M // 2
                DNF_min_or_upper = M
            elif ensemble_type == "symmetric":
                k = n // 2
                f = lambda x: sum((x >> i) & 1 for i in range(k)) >= k // 2
                DNF_min_or_upper = 2**k
            elif ensemble_type == "LTF":
                weights = [random.randint(1, 10) for _ in range(n)]
                f = lambda x: sum(w * ((x >> i) & 1) for i, w in enumerate(weights)) >= sum(weights) // 2
                DNF_min_or_upper = len([w for w in weights if w > 0])

            T = truth_table(f, n)
            lambda_2_val = lambda_2(T)
            log_lambda_2 = math.log2(1 + lambda_2_val)
            log_DNF_min_or_upper = math.log2(1 + DNF_min_or_upper)
            results.append((log_lambda_2, log_DNF_min_or_upper))

```

```

if not results:
    return {
        "metric_name": "lambda_2 vs DNF_min",
        "metric_value": None,
        "instances_tested": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

log_lambda_2_vals, log_DNF_min_or_upper_vals = zip(*results)
slope = (sum(log_lambda_2_vals) - sum(log_DNF_min_or_upper_vals)) / len(results)
conjecture_holds = all(log_lambda_2 <= n + log_DNF_min_or_upper + 5 * math.log2(n+1) for n, l

return {
    "metric_name": "lambda_2 vs DNF_min",
    "metric_value": slope,
    "instances_tested": len(results),
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else f"Slope {slope} is outside expected range"
}

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] or [11, 23, 37, 53, 71]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(json.dumps({"TRIAL": {"seed": seed, **result}}))
        results.append(result["metric_value"])

    mean_slope = sum(results) / len(results)
    std_slope = math.sqrt(sum((x - mean_slope)**2 for x in results) / len(results))
    support_fraction = sum(1 for r in results if r is not None and all(r <= n + 5 * math.log2(n+1)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_slope} std={std_slope} support_fraction={support_fraction}")
    elif any(r is not None and all(r > n + 5 * math.log2(n+1) for n in [8, 10, 12, 14]) for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if result is not None)
        print(f"RESULT: FALSIFIED counterexample=\"Slope outside expected range\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

{"TRIAL": {"seed": 11, "metric_name": "lambda_2 vs DNF_min", "metric_value": 7.0276395303248895, "instances_tested": 0, "conjecture_holds": false, "counterexample": "mapping_undefined"},
{"TRIAL": {"seed": 23, "metric_name": "lambda_2 vs DNF_min", "metric_value": 7.00788077049084, "instances_tested": 0, "conjecture_holds": false, "counterexample": "mapping_undefined"},
{"TRIAL": {"seed": 37, "metric_name": "lambda_2 vs DNF_min", "metric_value": 7.00585013217493, "instances_tested": 0, "conjecture_holds": false, "counterexample": "mapping_undefined"},
{"TRIAL": {"seed": 53, "metric_name": "lambda_2 vs DNF_min", "metric_value": 7.041945775829029, "instances_tested": 0, "conjecture_holds": false, "counterexample": "mapping_undefined"},
{"TRIAL": {"seed": 71, "metric_name": "lambda_2 vs DNF_min", "metric_value": 7.017263172398402, "instances_tested": 0, "conjecture_holds": false, "counterexample": "mapping_undefined"},
RESULT: SUPPORTED mean=7.020115876243618 std=0.013375328575307719 support_fraction=1.0

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The bound is vacuous in the tested regime due to metric saturation combined with the $+5 \cdot \log_2(n+1)$ slack. Since $\text{asc}(T) - \text{desc}(T) = \sum_k T_k(2k - N + 1)$, one has $\lambda_2(f) \leq N^2/4 = 2^{(2n-2)}$, so $\log_2(1 + \lambda_2) \leq 2n$. For the bound to become non-trivial we need $2n > n + 5 \cdot \log_2(n+1)$, which only kicks in around $n \geq 28$; the test caps at n where $2^n \leq 16$ (i.e. $n \leq 4$) or uses 16 instances of unspecified n , far below threshold. Furthermore, uniform random f has $\lambda_2 \approx 2^{(1.5n)}$ and $\text{DNF}_{\min} \approx 2^{n/n}$, so random sampling n

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test reports SUPPORTED with only 16 instances per seed at unspecified n , falling far short of the pre-registered ≥ 840 instances across $n \in \{8, \dots, 20\}$ and four ensembles; the critic's saturation argument ($\log_2(1 + \lambda_2) \leq 2n$) makes the bound vacuous until $n \geq 28$) means the tested regime cannot probe potential failures. The pre-registered support condition is not unambiguously met. | next: Run the full pre-registered protocol at $n \in \{8, 10, 12, 14, 16, 18, 20\}$ with ≥ 30 seeds per $(n, \text{ensemble}, M)$ cell, explicit

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	187527
2	preregistration	claude_max	opus	0	0	7622
3	novelty	claude_max	opus	0	0	3784
4	novelty	claude_max	opus	0	0	8262
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14232
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15062
7	critic	claude_max	opus	0	0	31402
8	judge	claude_max	opus	0	0	7346

Totals: 0 input tokens, 0 output tokens, $\sim \$0.0000$ cost, 275238 ms total latency. Provider mix: {'claude_max': 6, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/8bf8c5c64dba.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/8bf8c5c64dba.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/8bf8c5c64dba.tar.gz` (if generated)